TECHNICAL ARCHITECTURE & IMPLEMENTATION SPECIFICATION

Customer Churn Prediction & Automated Retraining System

End-to-End Enterprise Machine Learning Platform: Feature Engineering, Multi-Model Benchmarking, MLOps Experimentation, Drift Detection, and Containerized Serving

Document Version	1.0.0 (Production Architecture)	**Technical Stack**	Python 3.14 · uv · FastAPI · Docker · MLflow · Streamlit
Document Scope	Project 1 Implementation Roadmap & System Architecture	**Execution Status**	**Approved — In Implementation**
Base Dataset	Telco Customer Churn (7,043 rows × 33 raw → 27 clean features)	**Last Updated**	October 2026 (Internal Engineering Spec)

EXECUTIVE SUMMARY & OBJECTIVE

This specification establishes the engineering architecture and implementation plan for an end-to-end customer churn prediction and automated retraining system. The platform evaluates classical and gradient-boosted tree ensembles with imbalance mitigation (comparing SMOTE against native algorithmic class weights, locking in native class weighting and tree ensembles for production), tracks all Optuna Bayesian tuning runs via MLflow, calibrates decision thresholds via a business Cost-Utility Matrix (C_{FN} = \$200, C_{FP} = \$20; optimal cutoff 0.12), computes global and local SHAP explanations aggregated by parent feature categories, monitors post-deployment drift via Population Stability Index (PSI) and rolling live ROC-AUC with exponential sample decay (0.85^{weeks_old}), serves real-time inferences through a containerized FastAPI microservice, and provides business stakeholders with a 5-page interactive Streamlit dashboard.

Table of Contents

Section	Specification Topic & Engineering Scope	Status
Section 1	Goal Description & Core Architectural Decisions (Models, Imbalance, Optuna, Cost Cutoff, SHAP, ADR 0001)	**Delivered**
Section 1.1 – 1.3	System Architecture Topology (Figure 1), Actor & Use Case Model (Figure 2), & End-to-End Data Flow (Figure 3)	**Delivered**
Section 2.1	Repository Directory Structure & Milestone 1: Exploratory Data Analysis & Preprocessing (Figure 4)	**Completed**
Section 2.2	Milestone 2: Multi-Model Training, MLflow Tracking, SHAP & ADR 0001 Retraining (Figures 5, 6 & 7)	**Completed**

Section	Specification Topic & Engineering Scope	Status
Section 2.3	Milestone 3: FastAPI REST Service, Real-Time SHAP Triage & Docker Containerization (Figures 8 & 9)	**Active (Next)**
Section 2.4 – 2.5	Milestone 4: 5-Page Streamlit Monitoring Dashboard & Milestone 5: Documentation & Handover	**Planned**
Section 3 & 4	Milestone-to-Deliverable Traceability Matrix & CLI / Container Verification Plan	**Active**

1. Goal Description & Core Architectural Decisions

Deliver an end-to-end, production-grade Customer Churn Prediction & Automated Retraining System for Project 1 encompassing seven core engineering pillars:

- Complete data pipeline, candidate benchmarking, Bayesian hyperparameter optimization, and holdout evaluation with cost-utility threshold optimization.
- Global and individualized model interpretability using SHAP `TreeExplainer` with parent categorical feature grouping.
- Closed-loop retraining simulation adhering to ADR 0001, utilizing the production model artifact with dual drift monitoring (Unsupervised Population Stability Index [PSI] and Supervised performance decay) and exponential sample decay retraining.
- High-performance REST prediction service using FastAPI and Pydantic v2, exposing health, single prediction, batch scoring, and real-time SHAP explanation endpoints.
- Production containerization via multi-stage Docker and Docker Compose.
- Stakeholder Streamlit dashboard for executive KPI tracking, drift monitoring, and retention triage.
- Thorough automated testing suite and comprehensive architectural documentation.

Dimension	Decision & Implementation	Status
Candidate Models	Tree-based ensembles (`RandomForestClassifier` , `LGBMClassifier` , `XGBClassifier`). LightGBM ranked #1 (ROC-AUC 0.8528 ± 0.0104). Deep learning (PyTorch/TF) excluded per spec to avoid overfitting tabular data.	**Delivered**
Class Imbalance	Native algorithmic class weighting (`class_weight='balanced'` , `scale_pos_weight`). SMOTE excluded from production pipeline to avoid synthetic distortion in sparse one-hot spaces.	**Delivered**
Hyperparameter Optimization	Bayesian Optimization using Optuna (`TPESampler` , `MedianPruner`) across candidates, tracking parameters, metrics, and artifacts to MLflow (`./mlruns`).	**Delivered**
Threshold Optimization	Cost-Utility Matrix (C_{FN} = \$200, C_{FP} = \$20) sweep over [0.01, 0.99]. Optimal cutoff: **0.12**, reducing expected business loss by 59.5% (**\$21,860.00** net savings; Recall: **93.05%**).	**Delivered**
Model Interpretability	SHAP `TreeExplainer` with one-hot dummy variables aggregated into parent categories (Contract, Internet Service, Payment Method). Summary, beeswarm, dependence, and waterfall plots.	**Delivered**

Dimension	Decision & Implementation	Status
Retraining Simulation (ADR 0001)	`notebooks/simulate_retraining.py` consumes production model artifact (`models/model.joblib`). Implemented Unsupervised PSI on weekly batches, Supervised decay trigger (rolling live AUC), and Exponential Sample Decay (0.85^{weeks_old}).	**Delivered**
Unified CLI	Console script `churn` with subcommands: `train` , `benchmark` , `evaluate` , `explain` .	**Delivered**
Serving API (Milestone 3)	FastAPI with Pydantic v2 validation. Endpoints: `GET /health` , `POST /predict` , `POST /predict/batch` , and `POST /explain` with real-time SHAP parent feature attributions.	**Active (Next)**
Containerization (Milestone 3)	Multi-stage `Dockerfile` (`python:3.14-slim` , non-root user, health check probes) and `docker-compose.yml` .	**Active (Next)**
Dashboard (Milestone 4)	Multi-page Streamlit dashboard (Overview KPIs, Benchmark & Evaluation, Drift Timeline & PSI Heatmap, Live Prediction & Triage, MLflow Experiment Viewer).	**Planned**

1.1 System Architecture Overview (Layered Component Topology)

Figure 1 illustrates the end-to-end layered topology reorganized into a wide four-stage horizontal architecture spanning Data Ingestion, Model Training & Evaluation, FastAPI Containerized Serving, and Closed-Loop Drift Monitoring.

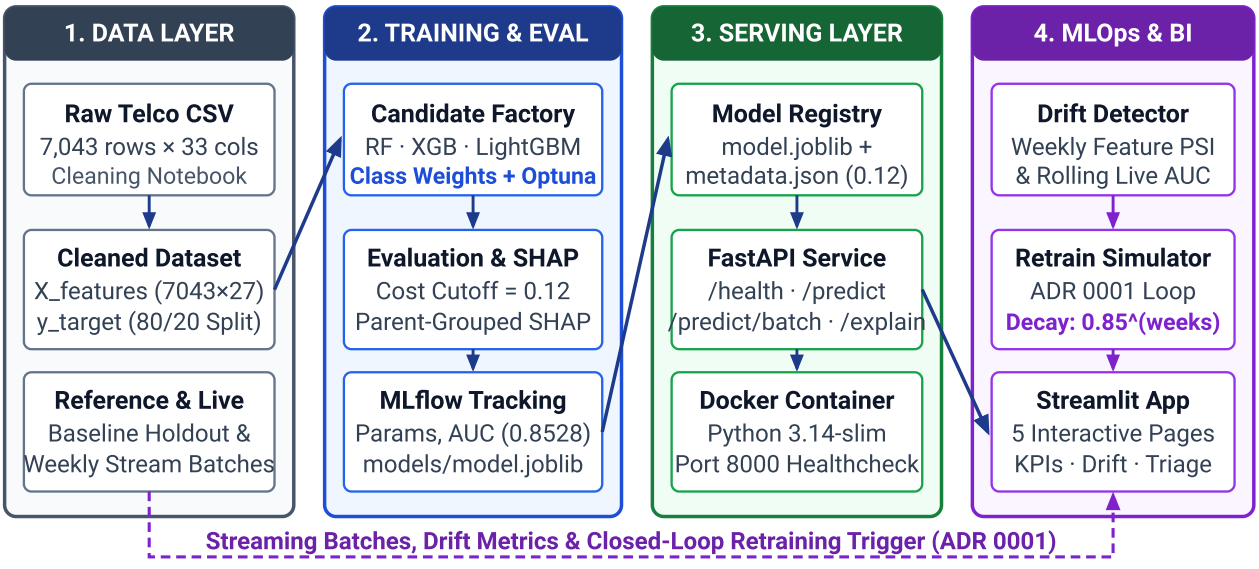


Figure 1: High-Level System Architecture & 4-Stage Layered Component Topology

1.2 System Actor & Use Case Model

Figure 2 presents the system actor and use case topology with operational and automated actors positioned on the left flank and analytical and engineering roles positioned on the right flank to eliminate connector crossover.

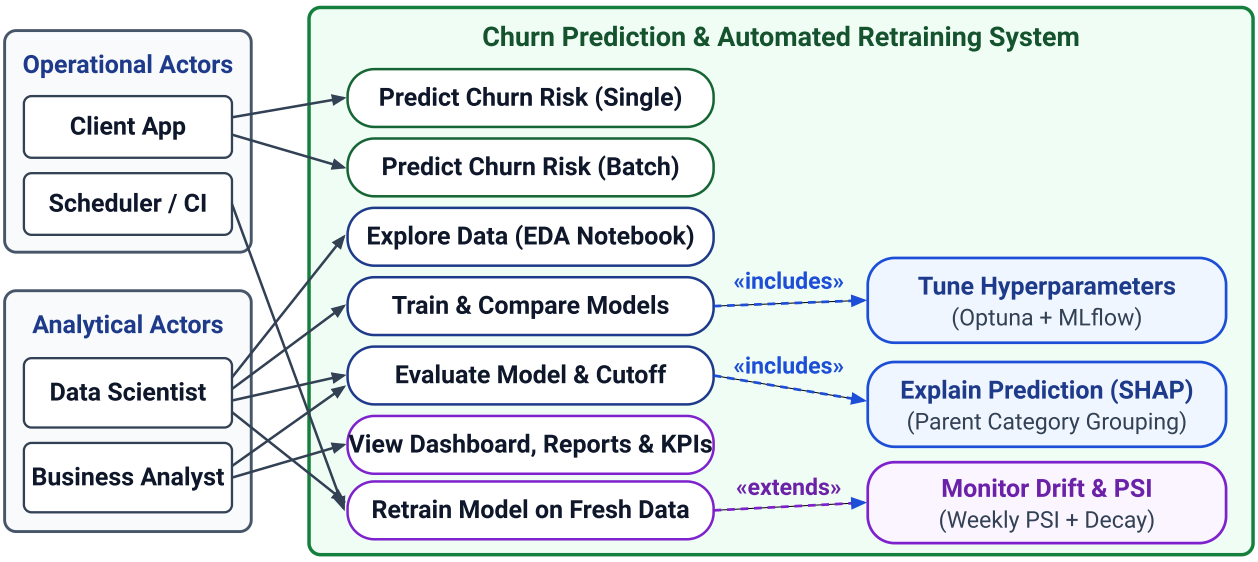


Figure 2: Actor & Use Case Model Across Engineering, Analyst, Scheduler & Client Roles

1.3 End-to-End Data Flow & Multi-Model Benchmarking Diagram

Figure 3 replaces the overlapping point-to-point connector grid with a clean, bus-routed horizontal pipeline showing data ingestion, cleaning, stratified splitting, dual imbalance handling (highlighting the production class-weighting path), candidate model training, and artifact generation.

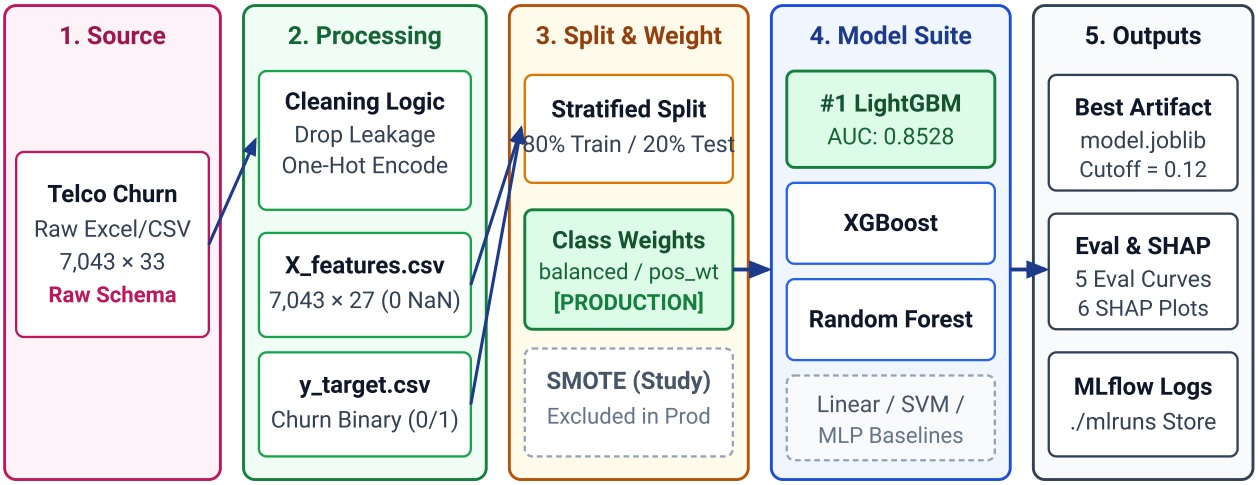


Figure 3: End-to-End Data Processing, Splitting & Multi-Model Benchmarking Pipeline

2. Detailed Milestone Status, Directory Structure & Roadmap

```
Final Project/
├── data/
│   ├── processed/                # Source of truth: X_features.csv, y_target.csv
│   ├── live/                     # Simulated streaming batches (features,
labels, pending)
│   ├── reference/                # Static pre-drift holdout baseline
│   ├── drift/drift_metrics.csv   # Simulation metrics (PSI, AUC, triggers)
│   └── retraining/week_N/        # Artifacts from simulation retraining triggers
├── models/
│   ├── model.joblib              # Serialized production model artifact
│   └── metadata.json             # Architecture, CV score, optimal threshold
(0.12)
├── reports/
│   ├── evaluation_report.md      # Holdout evaluation markdown report
│   └── figures/                  # 5 evaluation curves + 6 SHAP visualizations
├── src/final_project/
│   ├── __init__.py               # Public API exports
│   ├── data.py                   # Stratified data loading and splitting
│   ├── models.py                 # Candidate model factory & cross-validation
│   ├── tuning.py                 # Optuna Bayesian optimization & MLflow
integration
│   ├── evaluation.py             # Holdout evaluation, cost matrix optimization
│   ├── explainability.py         # SHAP TreeExplainer & parent feature grouping
│   ├── drift.py                  # Population Stability Index & exponential
sample decay
│   ├── pipeline.py               # High-level training orchestration
│   ├── cli.py                    # Unified churn CLI (train, benchmark,
evaluate, explain)
│   ├── serving/                  # [MILESTONE 3]
│   │   ├── __init__.py
│   │   ├── api.py                # FastAPI application
│   │   └── schemas.py            # Pydantic v2 input/output schemas
│   └── dashboard/                # [MILESTONE 4]
│       └── app.py                # Streamlit interactive application
├── tests/                        # Automated test suite (29 tests passing)
│   ├── __init__.py
│   ├── test_data.py
│   ├── test_models.py
│   ├── test_candidates.py
│   ├── test_tuning.py
│   ├── test_pipeline.py
│   ├── test_pipeline_tuning.py
│   ├── test_evaluation.py
│   ├── test_evaluation_cv.py
│   ├── test_explainability.py
│   ├── test_drift_monitoring.py
│   ├── test_simulator_integration.py
│   ├── test_cli.py
│   ├── test_cli_benchmark.py
│   ├── test_cli_explain.py
│   └── test_api.py               # [MILESTONE 3]
├── notebooks/
│   ├── [redacted]
```

2.1 Milestone 1: Data Collection, EDA & Preprocessing — [COMPLETED]

- **Status:** Complete & Verified.
- **Deliverables:**
  - `data/processed/X_features.csv` (7,043 × 27, all numeric, 0 NaNs).
  - `data/processed/y_target.csv` (Churn Value: 5,174 retained / 1,869 churned).
  - Cleaning logic documented inline in `notebooks/cleaning_for_retraining.ipynb`.

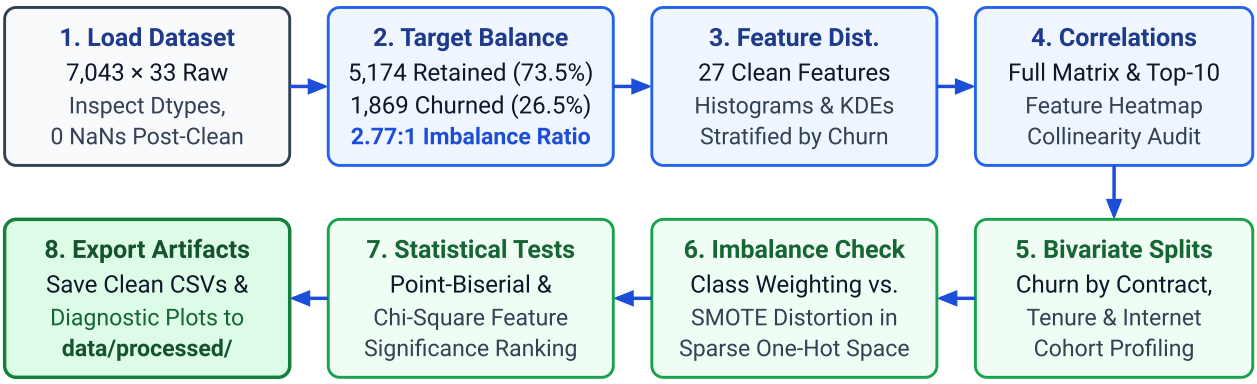


Figure 4: Exploratory Data Analysis (EDA) 8-Stage Execution Flow & Validation Pipeline (Wide 2-Row Serpentine Layout)

2.2 Milestone 2: Production Churn Modeling Pipeline, Evaluation & Explainability — [COMPLETED]

Status: 100% Implemented & Tested (PR #10 ready for review, 29/29 tests passing).

- **Tracer Bullets Delivered:**
  1. **#2 Baseline Pipeline Tracer Bullet:** Core seams (`final_project.data`, `models`, `pipeline`), stratified 80/20 splitting, baseline Random Forest with 5-fold Stratified CV, artifact serialization, and `churn train` CLI.
  2. **#3 Multi-Model Candidates & Class Weighting:** Unified candidate factory (`RandomForestClassifier`, `LGBMClassifier`, `XGBClassifier`) with native imbalance handling (`balanced`, `scale_pos_weight`). Exposed `churn benchmark` CLI (LightGBM ranked #1 with ROC-AUC 0.8528).
  3. **#4 Bayesian Hyperparameter Optimization & Experiment Tracking:** Optuna Bayesian search with `MedianPruner`, MLflow tracking backend (`mlruns`), and `churn train --tune`.
  4. **#5 Holdout Evaluation & Cost-Utility Thresholding:** Evaluation on unseen 20% test split (N = 1,409). Cost matrix (C_{FN} = \$200, C_{FP} = \$20) optimization identified optimal decision cutoff **0.12**, saving **\$21,860.00** in net business loss (59.5% reduction; Recall: 93.05%). Exported `reports/evaluation_report.md` and 5 diagnostic plots. Exposed `churn evaluate`.
  5. **#6 Model Interpretability with SHAP:** `TreeExplainer` exact Shapley values. One-hot dummy features aggregated into parent categories (Contract, Internet Service, Payment Method). Summary, beeswarm, dependence plots, and individual waterfall plots. Exposed `churn explain`.

- 6. **#7 Retraining Simulator Integration (ADR 0001):** Replaced toy logistic regression with serialized production model artifact. Added Unsupervised Data Drift Monitor calculating Population Stability Index (PSI) on incoming weekly batches, and Exponential Sample Decay (0.85^{weeks_old}) for adaptive retraining.

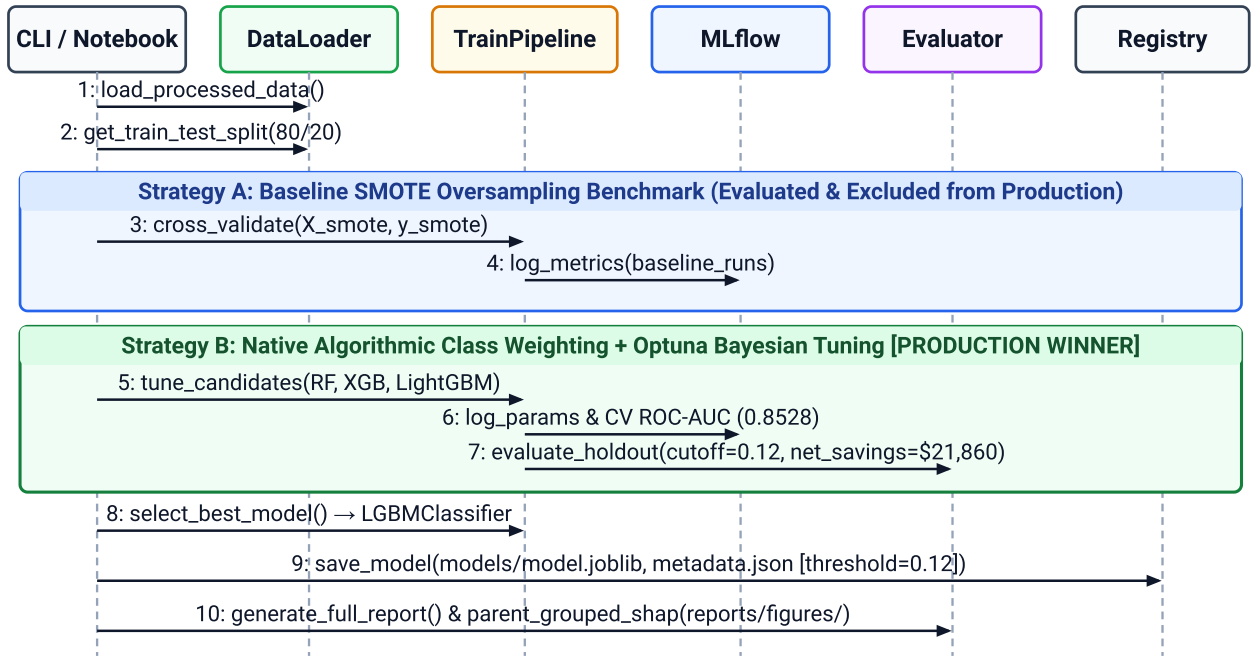


Figure 5: Dual-Strategy Benchmark, Optuna Bayesian Tuning & MLflow Logging Sequence (High-Contrast Layout)

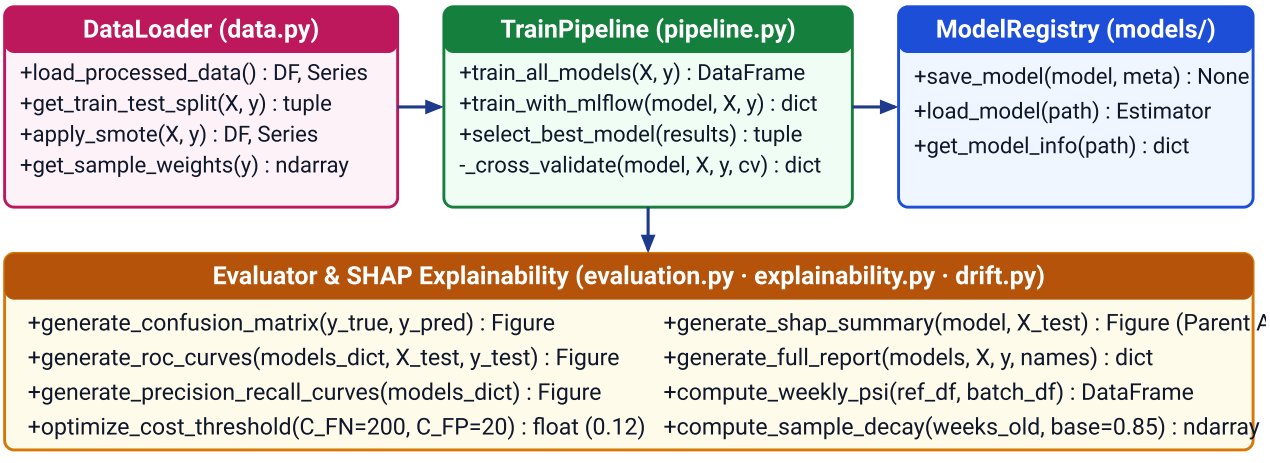


Figure 6: Data, Training, Evaluation, Explainability & Model Registry Class Architecture

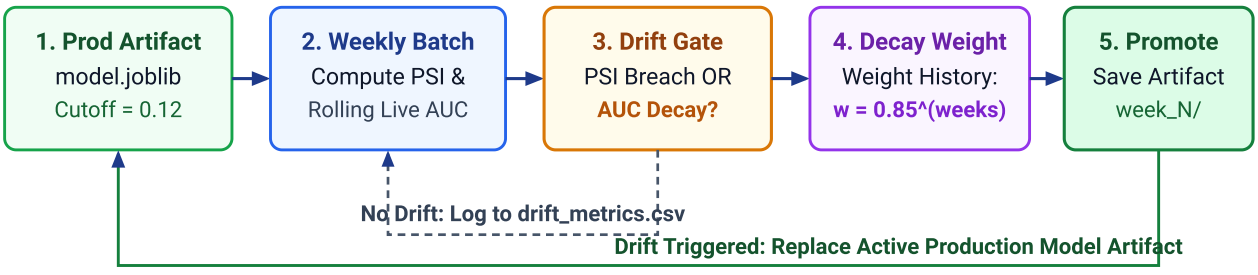


Figure 7: Closed-Loop Retraining Decision State Machine (ADR 0001 Verified)

2.3 Milestone 3: Production Prediction Service & Containerized Deployment — [ACTIVE / NEXT FRONTIER]

- **Status:** Ready for implementation.
- **Objectives:**
  - Build a high-performance REST prediction service using FastAPI and Pydantic v2.
  - Expose operational endpoints:
    - `GET /health` — service status, model architecture, metadata version.
    - `POST /predict` — single-customer scoring returning churn probability, risk category, and binary decision based on the cost-optimal cutoff (**0.12**).
    - `POST /predict/batch` — bulk scoring for tabular datasets.
    - `POST /explain` — on-demand SHAP parent feature attributions for frontline retention triage.
  - Package into a production-grade multi-stage `Dockerfile` (non-root security, health checks) and `docker-compose.yml`.

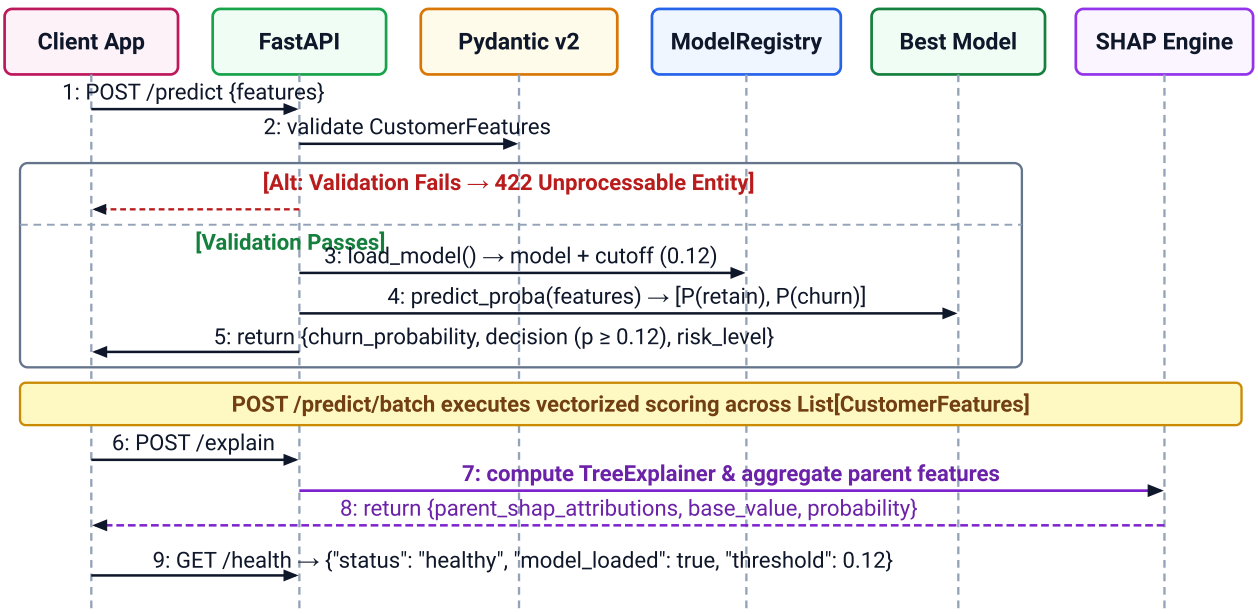


Figure 8: FastAPI Request Validation, Prediction, Real-Time SHAP Attribution & Health Check Sequence

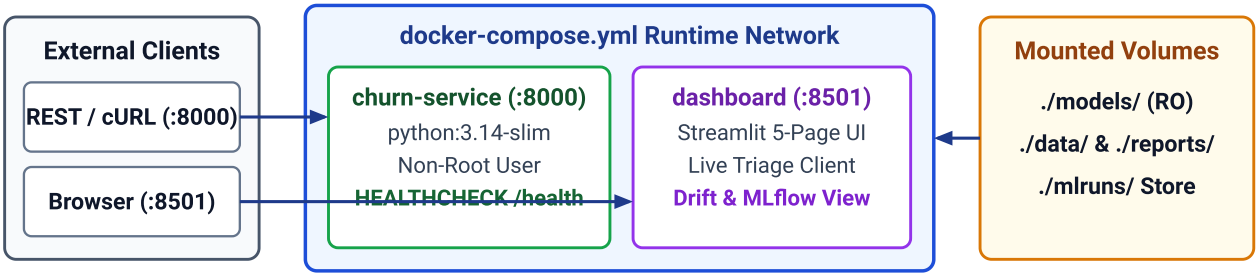


Figure 9: Multi-Stage Docker Container & Volume Deployment Topology (Milestone 3 & 4)

2.4 Milestone 4: MLOps, Monitoring & Web Dashboard — [PLANNED]

- **Status:** Scheduled following Milestone 3.
- **Objectives — Multi-Page Interactive Streamlit Dashboard** (`src/final_project/dashboard/app.py`):
  - **Page 1: Executive KPI Overview** — dataset metrics, churn rates, financial impact of threshold optimization ($21,860 net savings).
  - **Page 2: Model Benchmark & Diagnostics** — ROC curves, PR curves, calibration diagrams, confusion matrices across candidate models.
  - **Page 3: Drift & Retraining Monitor** — live timeline of rolling AUC, Population Stability Index (PSI) heatmaps, retraining events.
  - **Page 4: Live Retention Scoring & Triage** — interactive form to input customer attributes, receive churn risk gauge, and view real-time SHAP waterfall charts.
  - **Page 5: MLflow Experiment Hub** — embedded trial parameters, loss curves, and artifact exploration.

2.5 Milestone 5: Documentation & Presentation — [PLANNED]

- **Status:** Scheduled for final project wrap-up.
- **Deliverables:**
  - Comprehensive `README.md` with system diagrams, quick-start guide, and performance benchmarks.
  - Architecture Decision Records (`docs/adr/`).
  - Executive presentation slide deck summarizing methodology, financial optimization findings, and operational deployment.

3. Milestone-to-Deliverable Traceability Matrix

Status Legend: Completed & Verified (Milestones 1 & 2) · Active Frontier (Milestone 3) · Planned (Milestone 4) · Documentation (Milestone 5).

Milestone	Primary Code & Artifact Deliverables	Verification & Test Coverage	Status
Milestone 1: Data & EDA	`data/processed/X_features.csv` (7,043×27), `y_target.csv`, `cleaning_for_retraining.ipynb`	`tests/test_data.py` (0 NaNs, exact schema & stratified 80/20 split verified)	**Completed & Verified**
Milestone 2: Modeling, Eval & Drift	`models.py`, `tuning.py`, `evaluation.py`, `explainability.py`, `drift.py`, `cli.py`, `models/model.joblib`	29/29 unit & simulator integration tests passing; LightGBM 0.8528 AUC; 0.12 cutoff	**Completed & Verified**
Milestone 3: FastAPI & Docker	`src/final_project/serving/api.py`, `schemas.py`, `Dockerfile`, `docker-compose.yml`	`tests/test_api.py`, `/health`, `/predict`, `/predict/batch`, `/explain` probes	**Active Frontier**
Milestone 4: Streamlit BI	`src/final_project/dashboard/app.py` (5 interactive monitoring & triage pages)	End-to-end UI rendering of KPIs, ROC/PR plots, PSI heatmaps & live SHAP triage	**Planned**
Milestone 5: Docs & ADR	`README.md`, `docs/adr/0001-*.md`, Executive Presentation Deck	Architecture review & reproducibility walkthrough from clean clone	**Documentation**

4. Verification Plan & CLI Execution Suite

Automated Test Suite & CLI Execution

```
# Run complete test suite (29 existing tests + new API tests)
uv run pytest -v

# Run candidate model benchmark across 5 stratified folds
uv run churn benchmark --data-dir data/processed --cv-folds 5

# Run Bayesian hyperparameter optimization study
uv run churn train --data-dir data/processed --output-dir models --tune --n-trials
20

# Run holdout evaluation and cost-utility threshold optimization
uv run churn evaluate --data-dir data/processed --model-path models/model.joblib -
-report-dir reports

# Run SHAP explainability generation (with parent feature aggregation)
uv run churn explain --data-dir data/processed --model-path models/model.joblib --
output-dir reports/figures

# Run retraining simulator with dual drift monitoring (ADR 0001)
uv run python notebooks/simulate_retraining.py
```

Milestone 3 Verification (FastAPI Service & Docker Container)

```
# 1. Run FastAPI service locally
uv run uvicorn final_project.serving.api:app --host 0.0.0.0 --port 8000 --reload

# 2. Check service health & metadata
curl -X GET http://localhost:8000/health

# 3. Test single-customer prediction (verifying optimal 0.12 cutoff application)
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d @tests/fixtures/sample_customer.json

# 4. Test real-time SHAP triage attribution
curl -X POST http://localhost:8000/explain \
  -H "Content-Type: application/json" \
  -d @tests/fixtures/sample_customer.json

# 5. Build and run Docker container
docker build -t churn-prediction-api:latest .
docker run -d -p 8000:8000 --name churn-service churn-prediction-api:latest
curl http://localhost:8000/health
docker stop churn-service && docker rm churn-service
```